



Figure 10. Shared prompt example for machine translation. The examples are adopted from [5].

on the full prompt. Fig. 10 shows an example. Moreover, the shared prefix can be further tuned, via prompt engineering, to improve the accuracy of the downstream tasks [26, 27].

For this type of application, many user prompts share a prefix, thus the LLM service provider can store the KV cache of the prefix in advance to reduce the redundant computation spent on the prefix. In vLLM, this can be conveniently achieved by reserving a set of physical blocks for a set of predefined shared prefixes by the LLM service provider, as how OS handles shared library across processes. A user input prompt with the shared prefix can simply map its logical blocks to the reserved physical blocks (with the last block

context: (1) Which blocks should it evict? (2) How many blocks should it evict? (3) How to evict blocks if needed again? Typically, eviction policies use heuristics to predict which block will be accessed next in the future and evict that block. Since in vLLM we know that all blocks of a sequence are accessed together, we implement an all-or-nothing eviction policy, i.e., either all or none of the blocks of a sequence. Furthermore, multiple sequences within one request (e.g., beam search or one beam search request) are gang-scheduled as a *group*. The sequences within one sequence group can be preempted or rescheduled together due to potential sharing across those sequences. To answer the question of how to recover an evicted block, we consider several techniques:

Swapping. This is the classic technique used by many memory implementations which copy the evicted blocks to swap space on the disk. In our case, we copy evicted blocks to the CPU memory. As shown in Fig. 4, besides the block allocator, vLLM includes a CPU block allocator to manage the physical blocks swapped to CPU RAM. When the physical blocks swapped to CPU RAM are exhausted, it swaps free physical blocks for new tokens, it swaps